

SQL Questions

1. Employee Salary — Second Highest Salary

You have an `employees` table:

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	HR	45000
103	Arun	IT	75000
104	Meena	Finance	55000
105	Karthik	IT	70000

Task:

Write a SQL query to find the **second highest salary** from the `employees` table.

2. Department Average Salary

Table: `employees`

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	HR	45000
103	Arun	IT	75000
104	Meena	Finance	55000
105	Karthik	IT	70000

Task:

Find departments whose **average salary is greater than 60,000**.

Expected columns:

`department`
`average_salary`

3. Customers With No Orders

Tables:

customers

customer_id	customer_name
1	Ravi
2	Priya
3	Arun
4	Meena

orders

order_id	customer_id	amount
101	1	5000
102	2	7000
103	1	3000

Task:

Find customers who **have never placed an order**.

Expected columns:

```
customer_id  
customer_name
```

4. Highest Salary in Each Department

Table: employees

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	HR	45000
103	Arun	IT	75000
104	Meena	Finance	55000
105	Karthik	IT	70000
106	Divya	HR	65000

Task:

Find the employee who has the **highest salary in each department**.

Expected columns:

department
emp_name
salary

Hint: Consider using a **window function**.

5. Employees Earning More Than Department Average

Table: employees

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	HR	45000
103	Arun	IT	75000
104	Meena	Finance	55000
105	Karthik	IT	70000
106	Divya	HR	65000

Task:

Find employees whose salary is **greater than the average salary of their own department**.

Expected columns:

emp_name
department
salary

6. Top 2 Highest Paid Employees Per Department

Table: employees

Task:

Find the **top 2 highest-paid employees** from each department.

Expected columns:

department
emp_name
salary
rank

Requirement:

Use `ROW_NUMBER()`, `RANK()`, or `DENSE_RANK()`.

7. Monthly Sales Analysis

Table: `orders`

order_id	order_date	customer_id	amount
101	2026-01-10	1	5000
102	2026-01-15	2	7000
103	2026-02-05	1	4000
104	2026-02-20	3	8000
105	2026-03-10	2	6000

Task:

Calculate the **total sales for each month**.

Expected columns:

`month`
`total_sales`

8. Customers With Above-Average Total Orders

Tables:

customers

`customer_id`
`customer_name`

orders

`order_id`
`customer_id`
`amount`

Task:

Calculate the total order amount for each customer and return only customers whose **total spending is greater than the average customer spending**.

Expected columns:

customer_id
customer_name
total_spent

Hint: This can be solved using a CTE + aggregate functions.

9. Find Consecutive Employee Salaries

Table: employees

emp_id	emp_name	salary
101	Ravi	50000
102	Priya	60000
103	Arun	70000
104	Meena	80000
105	Karthik	90000

Task:

Find employees whose salary is **higher than the previous employee's salary** based on emp_id.

Expected columns:

emp_id
emp_name
salary
previous_salary

Hint: Use LAG().

10. Department With Maximum Total Salary

Table: employees

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	HR	45000
103	Arun	IT	75000
104	Meena	Finance	55000
105	Karthik	IT	70000

emp_id	emp_name	department	salary
106	Divya	HR	65000

Task:

Find the department having the **highest total salary expenditure**.

Expected columns:

```
department
total_salary
```

Concepts: GROUP BY, SUM(), ORDER BY, LIMIT / subquery.

11. Duplicate Customer Records

Table: customers

customer_id	customer_name	email
101	Ravi	ravi@gmail.com
102	Priya	priya@gmail.com
103	Arun	ravi@gmail.com
104	Meena	meena@gmail.com

Task:

Find all email addresses that are registered for **more than one customer**.

Expected columns:

```
email
customer_count
```

Hint: Use GROUP BY and HAVING.

12. Employees Joined in the Same Year

Table: employees

emp_id	emp_name	department	joining_date
101	Ravi	IT	2022-05-10
102	Priya	HR	2023-06-15
103	Arun	IT	2022-08-20
104	Meena	Finance	2024-01-10
105	Karthik	IT	2023-03-12

Task:

Find the number of employees who joined in each year.

Expected columns:

```
joining_year
employee_count
```

13. Products Never Sold

Tables:

products

product_id	product_name	price
1	Laptop	60000
2	Mouse	1000
3	Keyboard	2000
4	Monitor	15000

sales

sale_id	product_id	quantity
101	1	2
102	2	5
103	1	1

Task:

Find products that have **never been sold**.

Expected columns:

```
product_id
product_name
```

14. Calculate Product Revenue

Tables:

products

product_id	product_name	price
1	Laptop	60000
2	Mouse	1000
3	Keyboard	2000

sales

sale_id	product_id	quantity
101	1	2
102	2	5
103	1	1
104	3	4

Task:

Calculate the **total revenue generated by each product**.

Formula:

Revenue = price × quantity

Expected columns:

product_name
total_quantity
total_revenue

15. Employee and Manager Details

Table: employees

emp_id	emp_name	manager_id
101	Ravi	NULL
102	Priya	101

emp_id	emp_name	manager_id
103	Arun	101
104	Meena	102
105	Karthik	102

Task:

Display each employee along with their **manager's name**.

Expected columns:

employee_name
manager_name

Hint: Use a **self join**.

16. Find Employees Without Managers

Table: employees

emp_id	emp_name	manager_id
101	Ravi	NULL
102	Priya	101
103	Arun	101
104	Meena	102
105	Karthik	NULL

Task:

Find employees who **do not have a manager**.

Expected columns:

emp_id
emp_name

17. Count Employees Reporting to Each Manager

Table: employees

emp_id	emp_name	manager_id
101	Ravi	NULL
102	Priya	101
103	Arun	101
104	Meena	102
105	Karthik	102

Task:

Find the number of employees reporting directly to each manager.

Expected columns:

manager_id
employee_count

Hint: Use GROUP BY.

18. Find Missing Employee IDs

Table: employees

emp_id	emp_name
101	Ravi
102	Priya
104	Arun
105	Meena

Task:

Find the missing employee ID between the minimum and maximum employee IDs.

Expected result:

103

Hint: Think about LEAD(), LAG(), or a number-generating approach.

19. Find Employees With Same Salary

Table: `employees`

<code>emp_id</code>	<code>emp_name</code>	<code>salary</code>
101	Ravi	50000
102	Priya	60000
103	Arun	50000
104	Meena	70000
105	Karthik	60000

Task:

Find salaries received by **more than one employee**.

Expected columns:

`salary`
`employee_count`

20. Find Employees With Unique Salaries

Using the same `employees` table:

Task:

Find employees whose salary is **not shared by any other employee**.

Expected columns:

`emp_id`
`emp_name`
`salary`

Hint: Use `GROUP BY` with a subquery or window function.

21. Running Total of Sales

Table: `sales`

<code>sale_id</code>	<code>sale_date</code>	<code>amount</code>
101	2026-01-01	5000
102	2026-01-05	3000
103	2026-01-10	7000

sale_id	sale_date	amount
104	2026-01-15	4000

Task:

Calculate the **running total of sales** ordered by `sale_date`.

Expected columns:

```
sale_date
amount
running_total
```

Hint: Use `SUM()` `OVER()`.

22. Compare Current Sale With Previous Sale

Table: `sales`

sale_id	sale_date	amount
101	2026-01-01	5000
102	2026-01-02	7000
103	2026-01-03	6000
104	2026-01-04	9000

Task:

Display the current sale amount and the **previous day's sale amount**.

Expected columns:

```
sale_date
amount
previous_amount
```

Hint: Use `LAG()`.

23. Calculate Month-over-Month Sales Difference

Table: `monthly_sales`

month	total_sales
January	50000
February	65000
March	60000
April	80000

Task:

Calculate the difference between the **current month's sales and previous month's sales**.

Expected columns:

```
month
total_sales
previous_sales
sales_difference
```

24. Find First Order of Each Customer

Table: orders

order_id	customer_id	order_date	amount
101	1	2026-01-10	5000
102	1	2026-02-15	7000
103	2	2026-01-20	3000
104	2	2026-03-10	6000

Task:

Find the **first order placed by each customer**.

Expected columns:

```
customer_id
order_id
order_date
amount
```

Hint: Use `ROW_NUMBER()` or `MIN()`.

25. Find Latest Order of Each Customer

Using the same `orders` table.

Task:

Find the **most recent order** of every customer.

Expected columns:

```
customer_id
order_id
order_date
amount
```

Hint: Use `ROW_NUMBER()` with `ORDER BY order_date DESC`.

26. Customers With Multiple Orders

Table: `orders`

order_id	customer_id	amount
101	1	5000
102	1	7000
103	2	3000
104	3	6000
105	3	4000

Task:

Find customers who have placed **more than one order**.

Expected columns:

```
customer_id
order_count
```

27. Find the Third Highest Distinct Salary

Table: `employees`

emp_id	emp_name	salary
101	Ravi	90000
102	Priya	70000

emp_id	emp_name	salary
103	Arun	90000
104	Meena	60000
105	Karthik	80000

Task:

Find the **third highest distinct salary**.

Expected result:

70000

Requirement:

Duplicate salary values should be counted only once.

28. Find Products Above Average Price

Table: products

product_id	product_name	price
1	Laptop	60000
2	Mouse	1000
3	Monitor	15000
4	Keyboard	3000

Task:

Find products whose price is **greater than the average product price**.

Expected columns:

product_id
product_name
price

Hint: Use a subquery containing `AVG()`.

29. Find the Most Expensive Product in Each Category

Table: products

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	8000
4	Table	Furniture	15000
5	Monitor	Electronics	20000

Task:

Find the **most expensive product in each category**.

Expected columns:

```
category
product_name
price
```

Hint: Use `ROW_NUMBER()`, `RANK()`, or a subquery.

30. Find Departments With More Than 3 Employees

Table: employees

emp_id	emp_name	department
101	Ravi	IT
102	Priya	IT
103	Arun	IT
104	Meena	HR
105	Karthik	IT
106	Divya	HR
107	Sanjay	Finance

Task:

Find departments that have **more than 3 employees**.

Expected columns:

```
department
employee_count
```

Hint: Use `GROUP BY` and `HAVING`.

31. Employees Above Their Manager's Salary

Table: employees

emp_id	emp_name	salary	manager_id
101	Ravi	60000	NULL
102	Priya	70000	101
103	Arun	50000	101
104	Meena	80000	102
105	Karthik	65000	102

Task:

Find employees whose **salary is greater than their manager's salary**.

Expected columns:

emp_id
emp_name
employee_salary
manager_salary

Hint: Use a **self join**.

32. Department Salary Difference

Table: employees

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	IT	80000
103	Arun	HR	50000
104	Meena	HR	70000
105	Karthik	Finance	55000

Task:

For each department, find the difference between the **highest salary and lowest salary**.

Expected columns:

department
max_salary
min_salary
salary_difference

Hint: Use `MAX()` and `MIN()`.

33. Customers Who Purchased Every Product

Tables:

customers

customer_id	customer_name
1	Ravi
2	Priya
3	Arun

products

product_id	product_name
101	Laptop
102	Mouse
103	Keyboard

sales

customer_id	product_id
1	101
1	102
1	103
2	101
2	102
3	101

Task:

Find customers who have purchased **every available product**.

Expected columns:

customer_id
customer_name

Hint: Think about `COUNT (DISTINCT ...)`.

34. Find the Longest Employee Tenure

Table: employees

emp_id	emp_name	joining_date
101	Ravi	2018-05-10
102	Priya	2020-03-15
103	Arun	2016-08-20
104	Meena	2022-01-10

Task:

Find the employee who has been working for the **longest period**.

Expected columns:

emp_id
emp_name
joining_date

Hint: Use date difference and sorting.

35. Orders With Above-Average Order Amount

Table: orders

order_id	customer_id	amount
101	1	5000
102	2	12000
103	3	7000
104	1	15000
105	4	4000

Task:

Find all orders where the order amount is **greater than the overall average order amount**.

Expected columns:

order_id
customer_id
amount

Hint: Use a scalar subquery with `AVG ()`.

36. Category With the Most Products

Table: products

product_id	product_name	category
101	Laptop	Electronics
102	Mouse	Electronics
103	Keyboard	Electronics
104	Chair	Furniture
105	Table	Furniture
106	Pen	Stationery

Task:

Find the category that contains the **maximum number of products**.

Expected columns:

category
product_count

Hint: Use `GROUP BY`, `COUNT ()`, and a subquery.

37. Employees With No Salary Change

Table: employee_salary_history

emp_id	salary_date	salary
101	2025-01-01	50000
101	2025-06-01	50000
102	2025-01-01	60000

emp_id	salary_date	salary
102	2025-06-01	70000
103	2025-01-01	55000
103	2025-06-01	55000

Task:

Find employees whose salary **has not changed** between the recorded salary periods.

Expected columns:

emp_id
salary

Hint: Compare the minimum and maximum salary for each employee.

38. Detect Consecutive Login Days

Table: user_logins

user_id	login_date
101	2026-01-01
101	2026-01-02
101	2026-01-03
102	2026-01-01
102	2026-01-03
103	2026-01-05
103	2026-01-06

Task:

Find users who logged in on **at least 3 consecutive days**.

Expected columns:

user_id

Hint: Use date functions with `ROW_NUMBER()` or `LAG()`.

39. Find the Second Order of Each Customer

Table: orders

order_id	customer_id	order_date	amount
101	1	2026-01-01	5000
102	1	2026-02-10	7000
103	1	2026-03-15	3000
104	2	2026-01-05	6000
105	2	2026-02-20	9000

Task:

Find the **second order** placed by each customer.

Expected columns:

```
customer_id
order_id
order_date
amount
```

Hint: Use `ROW_NUMBER()` and filter for 2.

40. Calculate Percentage Contribution of Each Department

Table: employees

emp_id	emp_name	department	salary
101	Ravi	IT	60000
102	Priya	IT	80000
103	Arun	HR	50000
104	Meena	HR	70000
105	Karthik	Finance	40000

Task:

Calculate each department's **total salary** and its **percentage contribution to the company's total salary**.

Expected columns:

```
department
department_salary
total_company_salary
```

percentage_contribution

Hint: Use a window function such as:

SUM(salary) OVER ()

Advanced SQL Questions — Q41 to Q50

41. Department Hierarchy — Recursive CTE

Table: employees

emp_id	emp_name	manager_id
101	Ravi	NULL
102	Priya	101
103	Arun	101
104	Meena	102
105	Karthik	104
106	Divya	103

Task:

Display the complete **employee hierarchy**, starting from the top-level manager.

Expected columns:

emp_id
emp_name
manager_id
level

Example:

101	Ravi	NULL	1
102	Priya	101	2
104	Meena	102	3
105	Karthik	104	4
103	Arun	101	2
106	Divya	103	3

Hint: Use a **recursive CTE**.

42. Find Employees Whose Salary Increased Continuously

Table: `salary_history`

emp_id	salary_date	salary
101	2025-01-01	50000
101	2025-06-01	55000
101	2026-01-01	60000
102	2025-01-01	60000
102	2025-06-01	65000
102	2026-01-01	62000

Task:

Find employees whose salary **increased at every recorded salary change**.

Expected columns:

`emp_id`

Requirement:

Do not simply compare the first and last salary. Every intermediate salary must also increase.

Hint: Use:

`LAG ()`

and compare the current salary with the previous salary.

43. Top 3 Customers by Revenue in Each Year

Table: `orders`

order_id	customer_id	order_date	amount
101	1	2025-01-10	5000
102	2	2025-02-15	8000
103	1	2025-03-10	7000
104	3	2025-04-20	10000
105	2	2026-01-15	12000
106	1	2026-02-10	15000

Task:

Calculate each customer's yearly revenue and return the **top 3 customers for each year**.

Expected columns:

```
year
customer_id
total_revenue
rank
```

Hint: First aggregate then denserank

44. Find Missing Dates in Daily Sales

Table: `daily_sales`

sale_date	total_sales
2026-01-01	5000
2026-01-02	7000
2026-01-04	6000
2026-01-06	9000

Task:

Find all dates where **no sales record exists** between the minimum and maximum sale dates.

Expected result:

2026-01-03
2026-01-05

Hint: Generate a continuous date sequence using a **recursive CTE**, then compare it with `daily_sales`.

45. Find the Longest Consecutive Login Streak

Table: `user_logins`

user_id	login_date
101	2026-01-01
101	2026-01-02
101	2026-01-03
101	2026-01-06
102	2026-01-01
102	2026-01-02
102	2026-01-05

Task:

For each user, find their **longest consecutive login streak**.

Expected columns:

`user_id`
`longest_streak`

For example:

101 3
102 2

Hint: This is a classic **Gaps and Islands** problem.

Consider:

`ROW_NUMBER()`

combined with date arithmetic.

46. Customers Who Bought the Same Products

Table: `sales`

customer_id	product_id
101	1
101	2
101	3
102	1
102	2
102	3
103	1
103	2

Task:

Find pairs of customers who purchased **exactly the same set of products**.

Expected columns:

```
customer_1  
customer_2
```

Expected example:

```
101    102
```

Hint: Compare customers using:

```
COUNT(DISTINCT product_id)
```

and set-based aggregation.

47. Running Percentage of Total Sales

Table: `sales`

sale_date	amount
2026-01-01	5000
2026-01-02	10000
2026-01-03	15000
2026-01-04	20000

Task:

Calculate the **running sales amount** and the **running percentage of total sales**.

Expected columns:

sale_date
amount
running_total
running_percentage

Example concept:

sale_date	amount	running_total	running_percentage
Jan-01	5000	5000	10%
Jan-02	10000	15000	30%
Jan-03	15000	30000	60%
Jan-04	20000	50000	100%

Hint: Use two window calculations:

```
SUM(amount) OVER (...)
```

and

```
SUM(amount) OVER ()
```

48. Find Customers With Increasing Monthly Spending

Table: `customer_monthly_sales`

customer_id	month	amount
101	January	5000
101	February	7000
101	March	9000
102	January	8000
102	February	6000
102	March	10000

Task:

Find customers whose spending **increased every month**.

Expected result:

```
customer_id
101
```

Customer 102 should not be returned because:

8000 → 6000 → 10000

is not continuously increasing.

Hint: Use `LAG()` and verify every current amount is greater than the previous amount.

49. Find the Median Salary

Table: `employees`

emp_id	emp_name	salary
101	Ravi	40000
102	Priya	50000
103	Arun	60000
104	Meena	70000

emp_id	emp_name	salary
105	Karthik	80000

Task:

Find the **median salary** of all employees.

Expected result:

60000

Requirement:

Do not simply use `AVG()`.

Hint:

Use a window function such as:

`ROW_NUMBER()`

and determine the middle row(s).

For an even number of employees, calculate the average of the two middle salaries.

50. Identify Customers With a Complete Purchase Sequence

Tables:

customers

customer_id	customer_name
101	Ravi
102	Priya
103	Arun

orders

order_id	customer_id	order_date
1	101	2026-01-01

order_id	customer_id	order_date
2	101	2026-01-10
3	101	2026-01-20
4	102	2026-01-05
5	102	2026-01-25
6	103	2026-01-01

Task:

Find customers who placed **at least 3 orders**, and display the first order date, last order date, and number of days between them.

Expected columns:

```
customer_id
customer_name
order_count
first_order_date
last_order_date
days_between_orders
```

Hint: Use:

```
COUNT ()
MIN ()
MAX ()
DATEDIFF ()
```

with GROUP BY.